INTRODUCTION

Un algorithme est une suite d'instructions, qui une fois exécutée correctement, conduit à un résultat donné.

I- STRUCTURE GENERALE D'UN ALGORITHME

Pour écrire un algorithme nous avons besoins des mots clés ci-après : Algorithme, Var ou Variable, Const ou Constante, Début, Fin.

Syntaxe :

Algorithme nom_algorithme
Constante
    // Déclaration des constantes
Variable
    //Déclaration des variables
Début
    //Instruction 1
    //Instruction …
    //Instruction n
Fin

Remarque: Le nom d'un algorithme peut comporter des lettres et des chiffres, mais il exclut la plupart des signes de ponctuation, en particulier les espaces. Un nom d'algorithme correct commence impérativement par une lettre. Il est souhaitable d'utiliser un nom compréhensible qui n'est pas trop long.

II- LES VARIABLES

Une variable, c'est une petite information stockée en mémoire. Dans un algorithme, on va avoir en permanence besoin de stocker des valeurs. Ces valeurs peuvent être :

une ou des données fournies par l'utilisateur (saisies au clavier) ;
un ou des résultats obtenus après exécution d'une ou de plusieurs instructions de l'algorithme.

Exercice 1 : Identifier les valeurs que nous devons stocker pour écrire les algorithmes suivants :

1. Ecrire un algorithme qui affiche deux valeurs entières saisies au clavier par l'utilisateur ;
2. Ecrire un algorithme qui calcul la somme de deux entiers ;
3. Ecrire un algorithme qui calcul et affiche la somme de deux entiers ;
4. Ecrire un algorithme qui calcul et affiche le quotient et le reste de deux entiers ;
5. Ecrire un algorithme qui divise deux entiers ;
6. Ecrire un algorithme qui divise deux réels ;
7. Ecrire un algorithme qui affiche votre nom, votre prénom et votre âge ;
8. Ecrire un algorithme qui calcul votre âge en fonction de votre année de naissance.

Ces valeurs que nous stockons doivent être d'un type donné. En algorithmique, nous distinguons les types :

Entier: pour représenter les nombres entiers (1, 5, -9000, 1256, 98, -45)
Réel: pour représenter les nombres réels. (1.5, 5.0, -90.125, 1.256, 9.8, -45.0)
Caractère: Pour représenter un seul et unique caractère ("a", "b", " ", "7", "/", "^", "R", ".")
Une variable sexe aura pour valeurs "M" ou "F".
Chaîne de caractères: pour représenter une suite de caractères ("bonjour ", "Nous sommes au cours")
Booléen: un booléen est une variable qui ne peut prendre que deux valeurs : vrai ou faux.

Exemples :

"Aujourd'hui est un Lundi"

"Je suis une fille"

"J'ai les yeux bleus"

2+2=5

Toutes ces affirmations peuvent être considérées comme des variables booléennes. Si la dernière est toujours fausse, les valeurs logiques des trois autres varient suivant les cas.

Exercice 2 : reprendre l'exercice 1 et donnant le type de chaque variable identifiée.

Remarque: Les caractères et les chaîne de caractères sont toujours notée entre des doubles côtes pour éviter la confusion entre des nombres et des suites de chiffres.

124 est différent de "124". Le premier représente le nombre cent vingt-quatre qui est un entier et le second représente la suite de chiffres 1,2,4. On peut faire de calcul avec le premier mais le second non. Ceci permet aussi de ne pas confondre le nom des variables avec son contenu qui est la valeur prise par la variable.

Pour conclure, une variable est une valeur que nous stockons en mémoire et qui doit avoir un type et un nom unique. La valeur d'une variable peut être modifiée au cours de l'exécution de l'algorithme.

Dans un algorithme, il arrive qu'on ait besoins de stocker de l'information qui reste inchangée durant toute l'exécution de l'algorithme. On ne peut jamais modifier la valeur de cette information. Nous parlons dans ce cas de constantes.

Exercice 3 : Identifier le(s) variable(s) et constante(s) nécessaires pour écrire un algorithme qui calcule et affiche l'aire d'un cercle.

Calculer l'aire ou la surface d'un disque

Soit un disque de centre O et de rayon R. Son aire est égale à : $\pi \times R^2$. L'unité de l'aire du disque s'exprimera en unité au "carré" du rayon. Si le rayon est en cm, alors l'aire sera en cm2.

A. DECLARATION DES VARIABLES

La déclaration de variable consiste à indiquer le nom et le type de la variable. La première chose à faire avant de pouvoir utiliser une variable est donc de la déclarer en donnant :

Son nom: Le nom d'une variable peut comporter des lettres et des chiffres, mais il exclut la plupart des signes de ponctuation, en particulier les espaces. Un nom de variable correct commence impérativement par une lettre. Il est souhaitable d'utiliser un nom compréhensible qui n'est pas trop long.

Son type: Lorsqu'on déclare une variable, Il est obligatoire de préciser dès le départ ce qu'il contiendra. Cela permet à l'ordinateur de réserver en mémoire un espace correspondant au type de la variable (un entier aura besoin d'un petit espace, tandis qu'une chaine de caractères aura besoin d'un espace plus grand).

Nous devons déclarer les variables au début de l'algorithme, avant le mot clé « **Début** ».

Nous commençons la déclaration des variables par le mot clé « **Var** » ou « **Variable** ».

Syntaxe :

```
Variable
   nom_variable: type ;
```

Exemple

```
Variable
   nbre_Enfants : entier ;
   age : entier ;
   note : réel ;
```

Moyenne : réel ;
doublant : booléen ;
nom: chaine de caractère ;
prenom: chaine de caractère ;
sexe : caractère ;

Nous avons la possibilité de regrouper les variables en fonction de leur type tout en prenant le soin de les séparer par une virgule. On a donc :

Variable
nbre_enfants, age : entier ;
note, moyenne : réel ;
reponse : booléen ;
nom, prenom: chaine de caractère ;
sexe : caractère ;

De la même façon que nous déclarons les variables, les constantes doivent aussi être déclarées. Nous commençons la déclaration des constantes par le mot clé « Const » ou « **Constante** ».

Syntaxe :
Constante
Nom_constante = valeur;

Remarque: Il est recommandé de déclarer les constantes avant les variables.

Exercice 4 : Déclarer le (s) constante (s) et variable(s) nécessaires pour écrire l'algorithme qui calcul l'aire d'un cercle.

III- LES INSTRUCTIONS D'ENTREES / SORTIES

Imaginons que nous ayons écrit un algorithme qui calcul le carré d'un nombre, en supposant que ce nombre est égal à 12. On peut écrire:

```
Algorithme carre
Variable
    A : Entier ;
Début
    A ← 12*12 ;
Fin
```

D'une part, cet algorithme nous donne le carré de 12. Mais si l'on veut le carré d'un nombre autre que 12, il faut réécrire l'algorithme.

D'autre part, le résultat est calculé par la machine. Mais elle le garde soigneusement pour elle, et l'utilisateur qui fait exécuter cet algorithme, ne saura jamais quel est le carré de 12.

En algorithmique, Il existe des d'instructions pour permettre à la machine de dialoguer avec l'utilisateur. Un algorithme peut donc avoir des interactions avec l'utilisateur. Il peut :

afficher un résultat (du texte ou le contenu d'une variable).
demander à l'utilisateur de saisir une information afin de la stocker dans une variable.

Nous raisonnons ici en nous mettant "**à la place de l'ordinateur**".

1. L'instruction d'écriture

C'est une instruction de restitution de résultats sur le périphérique de sortie (en général l'écran).

Syntaxe :
Afficher (« Liste d'expressions ») ;
Ou bien
Ecrire (« Liste d'expressions ») ;
Exemple :

Afficher (x, y+2, "bonjour") ;

Quelle est la différence entre les deux algorithmes ci-dessous :

```
Algorithme salut1
Variable
   X, Y : entier ;
   T : chaine de caractère ;
Début
   X = 10 ;
   Y = 33 ;
   T = "bonjour" ;
   Afficher (x, y+2, "bonjour") ;
Fin
```

```
Algorithme salut2
Variable
   X, Y : entier ;
   T : chaine de caractère ;
Début
   X = 10 ;
   Y = 33 ;
   T = "bonjour" ;
   Afficher (x, y+2, T) ;
Fin
```

2. L'instruction de lecture

C'est une instruction de prise de données sur le périphérique d'entrée (en général le clavier).

Syntaxe :

Saisir (nom_variable1, nom_variable2)

Lire (nom_variable1, nom_variable2)

Exemple : Saisir (x);

Cette instruction affecte une valeur à la variable en prenant cette valeur sur le périphérique d'entrée. Dès que le programme rencontre une instruction « **Saisir** » ou « **Lire** », l'exécution s'interrompt, attendant la frappe d'une valeur au clavier.

Avant l'exécution de cette instruction, la variable avait ou n'avait pas de valeur. Après, elle a la valeur prise sur le périphérique d'entrée (le clavier).

Remarque : Avant de lire une variable, il est très fortement conseillé d'écrire des libellés à l'écran, afin de prévenir l'utilisateur de ce qu'il doit saisir.

Exemple : Écrire un algorithme qui demande à l'utilisateur de saisir son nom, son prénom et son âge qu'il affichera à l'écran.

```
Algorithme Affichage
Variable
     nom, prenom : chaine de caractère ;
     age : entier;
Début
     Ecrire ("Entrez votre nom, votre prénom et votre âge : ") ;
     Lire (nom, prenom, age);
     Ecrire ("Votre nom est : " nom ", votre prénom est : " prenom " et vous avez : " age " ans.") ;
Fin
```

EXERCICES

1-) Ecrire un algorithme qui demande à l'utilisateur de saisir un entier puis qui affiche le double de cet entier

```
Algorithme double
Variable
   Val, Dble : entier;
Début
```

```
   Ecrire ("Saisir un entier") ;
   Lire (Val) ;
   Dble ← Val * 2 ;
   Ecrire ("Le double de ", Val, " donne ", Dble ) ;
Fin
```

2-) Ecrire un algorithme qui demande à l'utilisateur de saisir un entier puis qui affiche le carré de cet entier

```
Algorithme Carre
Variable
   Val, Car : entier;
Début
   Ecrire ("Saisir un entier") ;
   Lire (Val) ;
   Car ← Val * Val ;
   Ecrire ("Le carré de ", Val, " donne ", Car) ;
Fin
```

IV- LES STRUCTURES DE CONTROLE

Un algorithme doit être capable de prendre des décisions. Pour y parvenir, les développeurs utilisent ce qu'on appelle des structures de contrôle. Ce nom cache en fait les deux éléments ci-dessous :

- les conditions : elles permettent d'écrire dans l'algorithme des règles comme « Si ceci arrive, alors fais cela » ;
- les boucles : elles permettent de répéter plusieurs fois une série d'instructions.

Savoir manier les structures de contrôle est fondamental.

A. LES CONDITIONS

Les conditions permettent d'exécuter une séquence d'instructions plutôt qu'une autre en fonctions de données venant de l'utilisateur, ou de résultats calculés faits dans l'algorithme.

Syntaxe

```
Si <condition> alors
   <séquence d'instructions>
finsi
```

<condition> est évaluée. Sa valeur peut être

vrai la séquence est exécutée

faux l'instruction est ignorée et on passe à l'instruction qui suit le finsi

Forme complète : instruction alternative :

Syntaxe

```
Si <condition> alors
   <séquence d'instructions 1 >
sinon
   <séquence d'instructions 2 >
Finsi
```

<condition> est évaluée. Sa valeur peut être

vrai la séquence 1 est exécutée

faux la séquence 2 est exécutée

Remarque : il faut que la condition soit évaluable, c'est à dire que les variables qui la composent éventuellement aient une valeur. Ci-dessous, une situation d'erreur

Début
  Si x > 5 alors
    …
  finsi
Fin

EXERCICES :

Ecrire un algorithme qui demande deux nombre à l'utilisateur, calcul et affiche le plus grand des deux

```
Algorithme maximum
Variable
   nbre1, nbre2, max : entiers ;
Début
   Afficher ('saisir deux entiers');
   Saisir (nbre1, nbre2);
   Si nbre1 > nbre2 alors
        max ← nbre1;
   Sinon
        max ← nbre2;
  finsi
  Afficher ('Le plus grand des deux nombres', nbre1, ' et ', nbre2, ' est ', maxi);
Fin
```

lignes	Nbre1	Nbre2	maxi	Ecran
2	10	15		
3 : faux			15	
5				
6				
7				Le plus grand des deux nombres 10 et 15 est 15